

AI-First Development

A Practitioner's Guide

Applicable to Claude Code, Windsurf, Cursor, and other AI coding assistants

Agenda

- 1. AI-First Philosophy
- 2. Addressing Common Concerns
- 3. Prompt Engineering vs Vibe Coding
- 4. Getting Started with Prompt Engineering
- 5. Scaling to Large Projects
- 6. Best Practices for Feature Documentation
- 7. Testing Strategies
- 8. Project Planning & Workflow
- 9. Git Best Practices
- 10. Tips & Tricks

1. AI-First Philosophy

What is AI-First Development?

- Traditional: Human writes code → AI assists occasionally
- AI-First: AI executes 100% → Human validates 100%

Key distinction:

- AI handles ALL coding, testing, documentation
 - Human handles ALL validation, review, decisions
 - Clear handoff points at each step
- Takeaway: Humans become architects & validators, AI becomes the builder

Core Development Principles

Specifications Drive Implementation:

- Database schema → API contracts → UI components
- Write specs BEFORE AI implements

Micro-Teams of 2:

- 2 humans + AI assistant = redundancy without overhead
- Each team owns end-to-end features

Own Your Stack:

- Be your own QA engineer
- Be your own DevOps engineer
- Own the entire vertical slice

The Paradigm Shift

From:

- "AI helps me write code"
- "Let me ask AI to fix this bug"
- "AI suggests completions"

To:

- "AI executes my specifications"
 - "I validate what AI produced"
 - "AI implements the full feature"
- Takeaway: Treat AI as your execution engine, not just an assistant

When AI-First Works Best

Ideal scenarios:

- New features with clear requirements
- CRUD operations and API endpoints
- Test generation and documentation
- Refactoring with defined patterns

Less ideal scenarios:

- Real-time pair programming
- Highly visual design work
- Hardware-specific debugging

2. Addressing Common Concerns

Common Concerns Engineers Raise

- 1. Code Quality & Reliability - Hallucinations, hidden bugs
- 2. Security Risks - Insecure patterns, data exposure
- 3. Maintainability - Opaque code, inconsistent style
- 4. Design Integrity - Architecture drift, loss of intent
- 5. Testing & Validation - False sense of coverage
- 6. IP & Compliance - License ambiguity
- 7. Developer Experience - Skill atrophy
- 8. Accountability - Who owns AI-generated bugs?

Code Quality & Security

The Concern:

- AI generates syntactically correct but logically flawed code
- AI might introduce vulnerabilities

How We Address It:

- Human validates 100% - every line is reviewed
 - Mandatory E2E tests catch integration issues
 - Pre-commit checks enforce quality gates
 - Three-layer review: Self → Automated → Peer
- Takeaway: Trust but verify - always review AI output

Maintainability & Design

The Concern:

- AI-generated code hard to understand
- May not follow team conventions
- Optimizes locally, not holistically

How We Address It:

- Project rules file defines conventions (CLAUDE.md, .windsurfrules)
 - Architecture documented upfront
 - Human writes specs BEFORE AI implements
 - AI follows existing patterns
- Takeaway: Document your standards where AI can read them

Testing, IP & Accountability

Testing:

- Test-first approach - define tests before implementation
- Human verifies test quality, not just coverage

IP & Compliance:

- Review AI suggestions for license issues
- Keep audit trail in commit history

Accountability:

- Human approves every PR = human owns the code
- Git history shows human review at each step
- Takeaway: Human approval = Human ownership

The Bottom Line

- AI-First $\neq$ AI-Only

The methodology addresses concerns through:

- Systematic validation at every step
 - Comprehensive testing (E2E + Unit)
 - Clear human ownership and accountability
 - Documented conventions in project rules
- Result: Faster development WITH maintained quality

3. Prompt Engineering vs Vibe Coding

What is Vibe Coding?

- Informal, exploratory approach with minimal instructions

Characteristics:

- Speed over precision
 - AI "guesses" intent
 - Minimal context provided
- Example: "Make something that sorts numbers"

Result:

- Quick for prototypes
- Unpredictable code quality

What is Prompt Engineering?

- Precise, structured inputs to guide AI behavior

Characteristics:

- Context and constraints provided
 - Clear success criteria
 - Examples included when helpful
- Example: "Write Python function to sort integers ascending, no built-in sort, return new list"
 - Result: Predictable, production-ready code

Side-by-Side Comparison

Vibe Coding

- Fast initial results
- Unpredictable quality
- Best for prototypes
- More rework later
- Hard to maintain

Prompt Engineering

- Slower initial setup
- Consistent quality
- Best for production
- Less rework overall
- Easy to maintain

When to Use Each

Use Vibe Coding for:

- Quick prototypes and experiments
- Exploring APIs or libraries
- Throwaway code

Use Prompt Engineering for:

- Production code
 - Team projects
 - Anything that will be maintained
 - Code that needs tests
- Takeaway: Default to prompt engineering for professional work

4. Getting Started with Prompt Engineering

Core Prompt Principles

Be Specific:

- Not "fix the bug" but "fix the race condition in token refresh"

Provide Context:

- Architecture, patterns, constraints
- Reference specific files

Define Success:

- What should work when done?
- What tests should pass?

- Takeaway: Treat prompts like specifications

Good vs Bad Prompts

■ Bad Prompts

- "Add search to the API"
- "Fix the authentication"
- "Make it faster"
- "Add validation"

■ Good Prompts

- "Add search: filter by email/name, case-insensitive, debounce 300ms"
- "Fix token refresh race condition when concurrent API calls"
- "Add pagination to /users endpoint, 20 per page, cursor-based"
- "Validate email format and password min 8 chars on registration"

Prompt Patterns

Feature Request:

■ Requirements → Technical Details → Success Criteria

Bug Fix:

■ Expected vs Actual → Steps to Reproduce → Error Message

Multi-Step:

■ Break into numbered steps

■ Define validation at each step

• Takeaway: Use templates for consistent results

Example: Well-Structured Prompt

PROMPT

I need to implement user authentication for my API.

Requirements:

- Users register with email/password
- JWT tokens for authentication
- Password hashing with bcrypt

API Endpoints:

POST /api/auth/register - Register new user

POST /api/auth/login - Login and get JWT

Success Criteria:

- Passwords never stored in plain text
- JWT expires after 1 hour
- E2E tests cover happy path and error cases

Please create the database migration first.

Common Prompt Mistakes

Over-reliance:

- "Build me a complete e-commerce platform"
- Fix: Break into smaller, specific requests

Under-specification:

- "Add validation"
- Fix: Specify what to validate and how

Forgetting Tests:

- "Implement user authentication"
- Fix: Add "Include E2E tests for..."

5. Scaling to Large Projects

The Challenge of Large Projects

When projects grow:

- 100+ tasks across modules
- Complex dependencies
- Context loss between sessions

The Solution:

- Phased development
- Centralized planning structure
- Continuous progress tracking

Project Planning Structure

Directory Layout:

- project/planning/ - Master plan and progress
- project/specs/ - Feature specifications
- project/sessions/ - Session summaries

Key Files:

- devplan.md - Master plan with phases and dependencies
 - devprogress.md - Progress tracker with checkboxes
 - database.md - Schema documentation
- Takeaway: Organize documentation for AI consumption

The Master Plan (devplan.md)

Contains:

- Phases with clear objectives
- Dependency mapping between features
- Tasks with checkboxes

Workflow per Feature:

- DB → API → Tests → UI → UI Tests
- Complete each layer before moving on
- Never skip testing

Progress Tracking (devprogress.md)

Update After Every Session:

- Mark completed tasks [x]
- Update phase percentages
- Document blockers

Quick Stats Table:

- Phase | Status | Progress
- ■ Not Started | ■ In Progress | ■ Complete

- Takeaway: Update progress religiously - it's your context for next session

When to Use What

Small Projects (<20 tasks):

- Simple project rules file is enough

Medium Projects (20-50 tasks):

- Add devplan.md and devprogress.md

Large Projects (50+ tasks):

- Full planning structure
- Session notes after every session
- Takeaway: Scale documentation with project complexity

6. Best Practices for Feature Documentation

The 5-Document Approach

- For large features, create 5 documents:

1. Planning Document (What & When):

- Timeline, scope, decisions, success criteria

2. Architecture Document (How):

- System diagrams, database schema, error handling

3. API Contracts (Interface):

- Endpoints, request/response examples, error codes

4. User Journey (Experience):

- Personas, step-by-step flows, edge cases

5. UI Wireframes (Visuals):

■ Page layouts, component specs, responsive design

Planning Document Structure

Key Sections:

- Related Documentation - links to other docs
 - Overview - problem statement and solution
 - Key Decisions - with rationale
 - Scope - what's in and explicitly out
 - Implementation Phases - with tasks
 - Critical Files - all files to create/modify
 - Success Criteria - measurable goals
- Takeaway: Explicit scope prevents scope creep

Architecture & API Contracts

Architecture Document:

- ASCII system diagrams showing component interactions
- Complete database schema with indexes
- Error handling strategies

API Contracts:

- Endpoint summary table
 - Request/response JSON examples
 - Error codes with descriptions
- Takeaway: AI produces better code with precise contracts

Design Review Process

Review Stages:

■ Draft → Review → Approved → Implement

Technical Review Focus:

■ Architecture - does it integrate cleanly?

■ API Design - RESTful and consistent?

■ Database - indexes sufficient?

■ Security - auth and validation complete?

• Takeaway: Catch issues early when changes are cheap

7. Testing Strategies

Modern Testing Philosophy

- Both E2E and Unit tests are MANDATORY

E2E Tests (Required):

- Complete user journeys
- API endpoint testing
- UI workflow testing

Unit Tests (Required):

- Business logic
- Utilities and helpers
- Edge cases and data transformations

Component Tests (When applicable):

- Microservices integration

■ Database operations with test containers

Test-First Approach

Before Implementation:

- Define coverage targets (70-90%)
- Build testing infrastructure first
- Document test scenarios

During Implementation:

- Write tests alongside code
 - Run tests frequently
 - Don't proceed if tests fail
- Takeaway: Tests are your regression safety net

Testing Best Practices

- Test user journeys, not implementation details
- Use data-test attributes for stable selectors

Coverage priorities:

- Happy path - must pass
 - Critical failures - invalid inputs, permissions
 - Edge cases - boundary conditions
 - Error scenarios - network failures, timeouts
- Run tests in pre-commit hooks
 - Takeaway: Tests that run automatically get run consistently

Coverage Measurement

Track coverage from all test types:

- Unit test coverage
- E2E test coverage
- Component test coverage (if applicable)

Merge coverage reports:

- See the complete picture
 - Identify gaps
- Takeaway: Measure coverage from ALL test types combined

8. Project Planning & Workflow

The 10-Step Development Process

- 1. Specification (Human writes detailed spec)
- 2. Database Schema (AI designs → Human reviews)
- 3. Repository Layer (AI implements → Human reviews)
- 4. API Endpoints (AI creates → Human reviews)
- 5. API E2E Tests (AI writes → Human verifies)
- 6. Frontend Components (AI builds → Human reviews)
- 7. UI E2E Tests (AI creates → Human verifies)
- 8. Documentation (AI updates → Human reviews)
- 9. Code Review (Human conducts)
- 10. Deployment (AI executes → Human verifies)

Handoffs & Quality Gates

Each Step Has:

- Clear deliverables
- Human approval checkpoint
- Tests that must pass

Quality Gates:

- Schema reviewed before repository
- API tests pass before frontend
- All tests pass before PR

- Takeaway: Never skip a step, never skip a review

AI vs Human Responsibilities

AI Executes

- Design database schema
- Implement repository layer
- Create API endpoints
- Write tests
- Build UI components
- Update documentation
- Execute deployment

Human Validates

- Write specifications
- Review all code
- Verify tests are meaningful
- Conduct code review
- Merge pull requests
- Make architectural decisions

9. Git Best Practices

Branch Naming & Commits

Branch Naming: /

- feature/user-auth
- fix/login-bug
- refactor/api-cleanup
- docs/readme-update

Conventional Commits: ():

- feat(auth): add Google OAuth login
- fix(api): handle null user gracefully
- test(auth): add E2E tests for login flow

- Takeaway: Consistent naming enables automation

Pre-Commit Checks (Mandatory)

Must Run Before Every Commit:

- Lint - code formatting and style
- Tests - unit and E2E tests
- Build - verify it compiles/bundles

Set Up Hooks:

- Use husky (Node) or pre-commit (Python)
- Fail commit if any check fails
- Takeaway: Catch issues locally, not in PR review

Hard Rules

- ■ Never commit directly to main
- ■ Never merge your own PR without review
- ■ Never commit code that fails tests
- ■ Never commit secrets (.env, credentials)
- ■ Never force push to main

AI Boundaries:

- ■ AI can: create branches, commit, push, open PRs
- ■ AI cannot: merge PRs, approve PRs
- Takeaway: Humans control what gets merged

10. Tips & Tricks

Communication Tips

Be Specific:

- "Fix the race condition in token refresh" not "fix the bug"

Reference Files:

- "In src/models/user.ts, add email_verified field"

Request Explanations:

- "Implement and explain the trade-offs"

Verify Understanding:

- "Before implementing, confirm: Goal is X, constraints are Y"

Productivity Techniques

Multi-Step Requests:

- Number your steps
- Define validation at each step

Batch Related Changes:

- "Update all error responses to use format X"

Progressive Refinement:

- Basic → Add feature → Add polish

Start Sessions with Context:

- "Continuing work on X. Last session we did Y."

Working with Large Codebases

Navigate:

- "Show me all files that handle authentication"

Understand First:

- "Explain how the current auth flow works"

Refactor Safely:

- "Refactor X to Y. Ensure all existing tests pass."

- Takeaway: Use AI to explore unfamiliar code

Pro Tips Summary

- Start sessions with context
- End sessions with notes (what was done, what's next)
- Use checkpoints - commit after each logical step
- Trust but verify - always review AI output
- Keep project rules file current
- Build a prompt library for your team
- Takeaway: Document what prompts work for your project

Summary & Action Plan

Key Takeaways

- ✓ AI-First: AI executes 100%, Human validates 100%
- ✓ Prompt Engineering > Vibe Coding for production
- ✓ Project rules file is essential
- ✓ 10-step workflow with clear handoffs
- ✓ E2E + Unit tests are both mandatory
- ✓ Pre-commit checks are non-negotiable
- ✓ Humans control what gets merged

Your Action Plan

Start Today:

- Create a project rules file for your current project

Next Feature:

- Use the 10-step process
- Write specs BEFORE AI implements

As Projects Grow:

- Add planning structure (devplan.md, devprogress.md)

Resources:

- Claude Code: claude.ai/code
- Windsurf: codeium.com/windsurf
- Cursor: cursor.sh

Thank You!
Questions?